

1. Organizzazione e struttura di un sistema operativo, principali funzionalità

- **Definizione:** Il Sistema Operativo (SO) è un insieme di software progettato per gestire l'hardware e le risorse software della macchina, fungendo da interfaccia tra l'utente e l'architettura fisica (hardware). Nasconde i dettagli tecnici complessi per presentare informazioni comprensibili all'uomo.
- **Funzionalità Principali:**
 - Controllo e gestione delle risorse hardware e dei processi di Input/Output (I/O) da e verso le periferiche.
 - Esecuzione dei programmi utente.
 - Gestione dell'archiviazione e dell'accesso ai dati tramite il File System.
 - Fornitura di un'interfaccia utente (grafica o testuale) per accedere alle risorse.
- **Struttura (Moduli principali):** Il SO è strutturato in componenti logiche precise, tra cui il **Kernel** (il cuore), lo **Scheduler**, il gestore della memoria e della sua protezione, il modulo di gestione I/O delle periferiche, il File System e lo spooler di stampa.
- **Prestazioni:** Il SO determina in modo diretto l'efficienza complessiva del sistema, impattando su stabilità, latenze di processamento e gestione dei crash.

2. Evoluzione dei sistemi operativi

L'evoluzione descritta dagli appunti evidenzia il passaggio da sistemi rigidi a sistemi interattivi e protetti:

- **Prima Generazione:** Caratterizzata dall'uso del puro linguaggio macchina e delle schede perforate.
- **Sistemi Batch Semplici (Seconda Generazione):** Introduzione dei linguaggi di alto livello e aggregazione dei programmi in lotti (batch) eseguiti in modo sequenziale. Il sistema carica un job in memoria e lo esegue fino alla fine prima di passare al successivo. **Limite (Warning):** Se un job richiede dati (attesa I/O), la CPU rimane totalmente inattiva in attesa della risposta della memoria, sprecando tempo di calcolo.
- **Sistemi Batch Multiprogrammati:** Per risolvere l'inefficienza dei batch semplici, il SO seleziona un sottoinsieme di job dal disco e li mantiene in memoria. Quando il job attivo si sospende in attesa di un evento o I/O, il SO riassegna immediatamente la CPU a un altro job, portando avanti più attività contemporaneamente.
- **Sistemi Time-Sharing (Sistemi a divisione di tempo):** Nascono per introdurre l'**interattività con l'utente** e la **multiutenza** (fornendo la percezione di una macchina virtuale dedicata a ciascun utente). Sfruttano il meccanismo del *context switch* per alternare i job sulla CPU: questo consente l'esecuzione di programmi molto lunghi senza bloccare il processore per gli altri utenti. Richiedono requisiti stringenti di scheduling della CPU, protezione della memoria e gestione dell'interazione diretta.

3. Passi del parsing della shell

La shell è un interprete di comandi evoluto che opera in un ciclo continuo. Prima di eseguire materialmente qualsiasi istruzione inserita dall'utente, esegue una fase fondamentale di **parsing (scansione)** strutturata nei seguenti passi:

1. **Lettura e Analisi Sintattica:** Scansione della stringa di comando inserita sul terminale.
2. **Identificazione dei Caratteri Speciali:** Individuazione di metacaratteri (come *, ?), simboli di ridirezione (<, >, >>) e operatori di piping (|) o di background (&).
3. **Sostituzione dei Metacaratteri:** Espansione dei nomi dei file (wildcard) presenti nella directory.

4. **Sostituzione delle Variabili:** Valutazione ed espansione delle variabili d'ambiente o locali (es. \$VARIABLE).
5. **Configurazione delle Ridirezioni:** Configurazione preventiva dei flussi di input/output prima di invocare l'effettivo programma.

4. Shell, principali comandi

- **Natura della Shell:** È un interprete di comandi che agisce tramite un ciclo continuo basato su tre fasi: lettura, interpretazione ed esecuzione delle istruzioni fornite dall'utente.
- **Classificazione dei Comandi:**
 - **Comandi Built-in:** Sono comandi integrati, eseguiti direttamente all'interno dello spazio della shell corrente (es. cd). Hanno la capacità di modificarne direttamente l'ambiente di esecuzione.
 - **Comandi Esterni (Eseguibili):** Sono veri e propri programmi binari residenti sul disco (es. ls, cat, cp). Per eseguirli, la shell deve istanziare un nuovo processo separato combinando le chiamate di sistema fork() ed exec1p().
- **Comandi Principali citati (nel file system UNIX):**
 - /bin e /sbin: Contengono i binari dei comandi essenziali di sistema e utente come cat, chmod (modifica permessi), cp (copia), ls (elenca file), mkdir (crea directory), pwd (mostra directory corrente), fdisk, reboot.

5. Definizione di processo, elementi principali, stati di un processo e relative system call

- **Definizione:** Un processo è un programma in esecuzione, caratterizzato da un'evoluzione sequenziale e concorrente all'interno di sistemi multiprogrammati.
- **Elementi Principali (Immagine del processo):** È composto dal codice (text), dai dati (variabili globali), dal Program Counter (PC) che indica la prossima istruzione, dai registri della CPU, dallo Stack (per parametri e variabili locali delle funzioni), dallo Heap (memoria dinamica) e dallo Stack del Kernel (utilizzato per gestire le System Call invocate dal processo).
- **Stati di un Processo (Ciclo di vita in UNIX):**
 - Init: Stato transitorio di creazione; il processo viene caricato in memoria e le sue strutture dati vengono inizializzate.
 - Ready: Il processo è pronto per essere eseguito ed è inserito in coda in attesa che lo scheduler gli assegni la CPU.
 - Running: Il processo ha il controllo della CPU e sta eseguendo le sue istruzioni.
 - Sleeping / Waiting: Il processo viene sospeso e si mette in attesa del verificarsi di un evento asincrono o del completamento di un'operazione di I/O.
 - Swapped: Stato tipico UNIX; il processo viene temporaneamente rimosso dalla memoria centrale e trasferito in memoria secondaria per liberare spazio.
 - Zombie: Il processo ha terminato la sua esecuzione, ma rimane registrato nel sistema in attesa che il processo padre ne rilevi lo stato di terminazione.
 - Terminated: Stato transitorio finale di deallocazione completa del processo dalla memoria.
- **Relative System Call di gestione:** fork() per la creazione, wait() per sincronizzarsi sulla terminazione del figlio, ed exit() per richiedere la terminazione volontaria.

6. Context switch, processi pesanti e processi leggeri

- **Context Switch (Cambio di contesto):** È l'operazione con cui il sistema operativo trasferisce il controllo della CPU da un processo \$P_i\$ a un processo \$P_{i+1}\$.

- **Fasi:** 1. Salvataggio dello stato corrente (registri, PC, ecc.) di SP_i all'interno del suo specifico Process Control Block (PCB). 2. Ripristino e caricamento dello stato del nuovo processo SP_{i+1} dal suo rispettivo PCB nei registri della CPU.
- **Overhead:** Questa operazione introduce un costo computazionale aggiuntivo (tempo in cui la CPU non esegue lavoro utile per l'utente) che dipende direttamente dalla frequenza dei cambi e dalla dimensione/complessità del PCB.
- **Processi Pesanti (Heavyweight Processes):** Corrispondono al concetto classico di processo. Possiedono uno spazio di indirizzamento privato e isolato e un PCB completo e separato. Il context switch tra processi pesanti è oneroso poiché richiede il cambio completo delle mappe di memoria.
- **Processi Leggeri (Lightweight Processes / Thread):** Sono unità di esecuzione interne a un processo. Adottano un modello ad ambiente globale: tutti i thread generati dallo stesso compito condividono il medesimo spazio di indirizzamento, il codice e i dati comuni. Mantengono private solo le risorse strettamente necessarie all'esecuzione, ossia il proprio Program Counter (PC), i registri e lo stack privato per gestire le proprie chiamate a funzione.
- **Impatto sul Context Switch:** Avendo strutture dati e PCB decisamente ridotti rispetto ai processi pesanti, il cambio di contesto tra thread dello stesso processo comporta un overhead nettamente inferiore.

7. Creazione processo Unix vs creazione thread Java

- **Creazione Processo UNIX:** Avviene tramite una duplicazione simmetrica basata sulla system call `fork()`. Il processo padre genera un figlio allocando una nuova *process structure* nella *process table* del Kernel. Il figlio eredita lo stesso codice del padre e ottiene una copia fisica separata e non condivisa dei segmenti di dati e dello stack del genitore. Per eseguire un programma differente, il figlio deve successivamente invocare una system call della famiglia `exec()`, sovrascrivendo interamente la propria immagine in memoria.
- **Creazione Thread (Modello Generale/Java rispetto a UNIX):** I thread vengono creati all'interno dello stesso spazio d'indirizzamento del processo chiamante. Non si ha una duplicazione della memoria o dei segmenti dati; il nuovo thread condivide istantaneamente le variabili globali, l'heap e i file aperti del processo che lo ha generato. Viene allocato solo uno stack privato minimale e un Program Counter indipendente. Il costo di creazione è estremamente più basso rispetto alla `fork()` UNIX.

8. Processi interagenti: sincronizzazione e comunicazione con relative system call

I processi si dicono interagenti se l'esecuzione di uno influenza l'altro (cooperazione o competizione). Le interazioni vengono gestite secondo due filosofie progettuali:

- **Ambiente Globale (Condivisione):** I processi o thread leggono e scrivono in un'area di memoria comune. Per evitare corruzioni dei dati, l'accesso alla *sezione critica* (porzione di codice che modifica variabili condivise) deve essere protetto garantendo la **Mutua Esclusione** tramite una *entry section* e una *exit section*.
 - **Semafori:** Sono tipi di dati astratti o variabili protette caratterizzate da un valore intero iniziale. Se il valore $SS=1$ agisce da Mutex (binario), se $SS>1$ è a conteggio. Quando un processo richiede una risorsa, se $SS>0$ il valore viene decrementato e si procede; se $SS=0$ il processo viene sospeso in una coda d'attesa. Al rilascio della risorsa, SS viene incrementato e un processo in coda viene risvegliato.
- **Ambiente Locale (Scambio di Messaggi):** La comunicazione avviene in modo esplicito senza memoria condivisa (Inter-Process-Communication, IPC). Può essere *diretta* (specificando

esplicitamente il nome del processo mittente/destinatario) o *indiretta* (depositando i messaggi in caselle postali/mailbox in logica multi-a-molti).

- **System Call pipe():** Consente di creare un canale di comunicazione unidirezionale funzionante come una coda FIFO a capacità limitata (circa 4096 byte). È utilizzabile solo tra processi in relazione di parentela (padre-figlio o fratelli) perché i file descriptor associati (es. `fd[0]` per la lettura e `fd[1]` per la scrittura) vengono ereditati durante la `fork()`.

9. Gestione segnali da parte di un processo, relative system call, segnali principali

- **Definizione:** I segnali sono vere e proprie interruzioni software (interrupt SW) inviate asincronamente a un processo per notificare il verificarsi di un determinato evento. Alla ricezione, l'esecuzione del processo viene interrotta istantaneamente per eseguire la reazione programmata, per poi riprendere (se il processo non viene ucciso) dalla stessa istruzione in cui era stato sospeso.
- **Meccanismo di Gestione (Handler):** Il programmatore può definire una funzione speciale detta *handler* per specificare come il programma debba reagire a un segnale. L'associazione tra segnale e handler avviene tramite system call:
 - `signal(int signum, void (*handler)(int))`: Associa direttamente una funzione handler a un determinato segnale.
 - `sigaction(int signum, const struct sigaction *act, struct sigaction *oldact)`: Versione evoluta e robusta della precedente. Permette di configurare una maschera di segnali (`sa_mask`) per bloccare temporaneamente la ricezione di altri segnali durante l'esecuzione dell'handler stesso, impedendo perdite di dati o annidamenti incontrollati.
- **Segnali Principali:**
 - **SIGINT (2):** Inviato tramite la combinazione da tastiera CTRL+C, richiede l'interruzione del processo attivo in foreground.
 - **SIGKILL (9):** Provoca la terminazione immediata e incondizionata del processo. È un segnale speciale ed inderogabile: non può essere né intercettato tramite un handler né ignorato dal processo.
 - **SIGCHLD (17):** Inviato in automatico dal Kernel a un processo padre nel momento esatto in cui un suo processo figlio termina. L'handler di questo segnale nel padre esegue tipicamente una chiamata a `wait()` per raccogliere lo stato del figlio ed evitare la permanenza di processi zombie nel sistema.
 - **SIGUSR1 (10) e SIGUSR2 (12):** Gli unici due segnali interamente personalizzabili messi a disposizione da UNIX per scopi di sincronizzazione custom scelti dal programmatore.

10. Immagine di un processo UNIX, cosa succede dopo fork/exec

- **Immagine di un Processo UNIX:** È l'insieme completo delle aree di memoria e delle strutture dati che il kernel associa a un processo attivo. Comprende:
 - **Process Structure:** Contiene i metadati vitali del processo (PID, stato, priorità, puntatori). È sempre residente nella memoria centrale (non swappable) all'interno della *Process Table* del Kernel.
 - **User Structure (Area U):** Contiene i dati necessari solo quando il processo è in esecuzione (registri CPU, tabella dei file descriptor aperti, handler dei segnali, directory corrente). È un'area swappable.
 - **Text:** Il codice sorgente binario ed eseguibile (sola lettura).
 - **Area Dati Globali:** Spazio per le variabili globali e statiche.

- *Stack e Heap*: Aree dinamiche; lo Stack cresce per gestire i record di attivazione delle funzioni e le variabili locali, lo Heap gestisce l'allocazione dinamica della memoria.
- *Stack del Kernel*: Area privata usata dal processo durante l'esecuzione in Kernel Mode (es. durante le system call).
- **Cosa succede dopo la `fork()`**: Il Kernel crea un processo figlio allocando una nuova voce nella *Process Table*. Copia interamente i segmenti di dati e lo stack del padre nei nuovi segmenti fisici allocati per il figlio. Padre e figlio condividono lo stesso segmento di testo (codice) e gli stessi file descriptor, ma hanno spazi di memoria separati e non condivisi (modifiche ai dati del figlio non toccano il padre).
- **Cosa succede dopo la `exec()` (es. `exec1p()`)**: Non viene creato alcun processo nuovo e il PID e il PID del padre rimangono inalterati. L'intera immagine in memoria precedente del processo (codice text, dati globali, stack e heap) viene **completamente sovrascritta e sostituita** dal codice e dai dati del nuovo programma letto dal disco.

11. Tipologie di scheduling

Lo scheduling è l'attività con cui il sistema operativo stabilisce l'ordine temporale di accesso alle risorse hardware (principalmente la CPU) massimizzando l'efficienza del sistema. Si suddivide in tre grandi tipologie operative a seconda dell'orizzonte temporale:

- **Scheduling a Lungo Termine (Long-Term Scheduling)**: Opera sulla memoria secondaria (disco) selezionando i programmi (job) sottomessi dagli utenti per caricarli nella memoria centrale (RAM), determinando la creazione formale dei relativi processi e controllando il grado di multiprogrammazione.
- **Scheduling a Medio Termine (Gestito dallo Swapper)**: Controlla il flusso dei processi tra la memoria centrale e la memoria secondaria. Interviene in situazioni di sovrallocazione della RAM eseguendo operazioni di:
 - *Swap-out*: Sospensione e trasferimento temporaneo di interi processi (o porzioni di essi) bloccati o molto pesanti sul disco per liberare spazio in RAM.
 - *Swap-in*: Reintroduzione in memoria centrale dei processi precedentemente salvati su disco non appena le risorse tornano disponibili.
- **Scheduling a Breve Termine (Short-Term Scheduling)**: Seleziona in modo estremamente rapido quale processo, tra quelli presenti in stato *Ready* all'interno della *ready queue*, debba ottenere l'assegnazione della CPU. La scelta politica dell'ordinamento è di competenza dello *scheduler*, mentre il meccanismo software che effettua materialmente il cambio di contesto e lo scambio sul processore prende il nome di **Dispatcher**.

12. Criteri e algoritmi di scheduling a breve termine

- **Criteri di Performance (Metriche di valutazione)**:
 - *Utilizzo della CPU*: Percentuale di tempo in cui la CPU esegue lavoro utile; deve essere massimizzato riducendo i tempi morti.
 - *Throughput (Produttività)*: Numero di processi portati a completo termine nell'unità di tempo.
 - *Turnaround (Tempo di completamento)*: Tempo totale che intercorre dal momento esatto in cui un processo viene sottomesso al sistema fino al suo completo termine.
 - *Tempo di attesa (Waiting Time)*: Somma del tempo totale trascorso da un processo all'interno della *ready queue* in attesa di ricevere la CPU.

- *Tempo di risposta*: Tempo necessario tra la sottomissione del processo e l'ottenimento della primissima risposta visibile.
- **Macro-classificazione delle politiche:**
 - *Senza diritto di prelazione (No Preemption)*: Una volta assegnata la CPU a un processo, questo la mantiene tassativamente finché non termina l'esecuzione o si blocca autonomamente per un'operazione di I/O.
 - *Con diritto di prelazione (Preemption)*: Il sistema operativo ha l'autorità di interrompere forzatamente il processo in esecuzione per sottrargli la CPU e assegnarla a un altro processo reputato più prioritario o per fine tempo.
- **Algoritmi Principali:**
 - **FCFS (First-Come, First-Served)**: Algoritmo senza prelazione basato su una coda FIFO rigorosa; serve i processi esattamente in base al loro ordine cronologico di arrivo.
 - **Round Robin (RR)**: Algoritmo con prelazione pensato per i sistemi interattivi. Assegna a ogni processo un quanto di tempo fisso di esecuzione (es. 20ms). Se il processo non termina entro il quanto, viene interrotto dal dispatcher, rimesso in coda e la CPU passa al processo successivo.
 - **SJF (Shortest Job First) / SRTF (Shortest Remaining Time First)**: Dà la precedenza assoluta al processo che richiede il minor tempo di servizio (o con il tempo residuo minore nel caso di SRTF con prelazione). Minimizza il tempo medio di attesa ma può causare il fenomeno della **Starvation** (i processi lunghi non vengono mai eseguiti se arrivano continuamente processi corti).
 - **Priority Scheduling (PS)**: Assegna la CPU basandosi su un valore di priorità associato a ogni processo.
 - **MLFQ (Multilevel Feedback Queue)**: Gestisce lo scheduling organizzando i processi in più code separate a priorità differenti, permettendo ai processi di spostarsi dinamicamente tra le code in base al loro comportamento storico (feedback).

13. 12 bit di protezione e come modificarli

- **Struttura della Protezione in UNIX**: La sicurezza e i permessi di accesso ai file sono regolati da un insieme di bit memorizzati direttamente all'interno del descrittore del file, ovvero l'**i-node**. Questi bit sono organizzati in una stringa logica strutturata su 12 bit complessivi:
 - **9 bit di permesso standard**: Suddivisi in tre gruppi da 3 bit ciascuno (rwx), regolano i permessi di Lettura (r), Scrittura (w) ed Esecuzione (x) rispettivamente per tre classi di utenti:
 - *Proprietario* (Owner / User).
 - *Gruppo* (Group).
 - *Altri* (Others).
 - **3 bit speciali**: Posizionati in testa, modificano il comportamento esecutivo del file:
 - **setuid**: Consente a un utente che esegue il file di assumere temporaneamente e per la sola durata dell'esecuzione i privilegi di sicurezza del proprietario effettivo del file (es. privilegi di root).
 - **setgid**: Funzionamento analogo al setuid, ma applicato ai privilegi del gruppo proprietario.
 - **Sticky bit**: Applicato storicamente alle directory (come la cartella temporanea /tmp), impedisce tassativamente agli utenti comuni di cancellare o rinominare file di

proprietà altrui, anche qualora possiedano permessi di scrittura completi sulla directory stessa.

- **Come Modificarli:** Per alterare la configurazione di questi bit di protezione si utilizza il comando di shell **chmod**. La modifica può essere impartita sia in modalità ottale numerica (es. `chmod 755 file.txt`) sia in modalità simbolica (es. `chmod u+s eseguibile`).

14. Metodi di allocazione di file in memoria secondaria e possibili metodi di accesso

- **Metodi di Accesso ai File (Interfaccia logica):**
 - *Accesso Sequenziale:* Il file viene processato come una sequenza ordinata di record (\$R_1, R_2, ..., R_n\$). Per accedere a un record specifico \$R_i\$, il sistema deve obbligatoriamente scorrere e leggere tutti i precedenti \$i-1\$ record in ordine.
 - *Accesso Diretto:* Il file viene concepito come un insieme di blocchi logici numerati. È possibile richiedere l'accesso immediato a un blocco qualsiasi specificandone semplicemente il numero identificativo. È fondamentale per le applicazioni che estraggono dati puntuali da grandi volumi (es. Database).
 - *Accesso a Indice:* Al file viene affiancata una struttura dati ausiliaria separata (l'indice) che contiene chiavi di ricerca e puntatori diretti alle locazioni delle informazioni nel file.
- **Metodi di Allocazione in Memoria Secondaria (Organizzazione fisica sul disco):**
 - *Allocazione Contigua:* Ogni file occupa un insieme di blocchi fisici sequenziali e adiacenti sul disco. **Vantaggi:** Velocità di ricerca massima (spostamento minimo della testina). **Svantaggi:** Genera forte frammentazione esterna ed è difficile prevedere la crescita futura di un file.
 - *Allocazione Concatenata (Linkata):* I blocchi associati a un file sono organizzati in una lista collegata sul disco. Ogni blocco contiene i dati utili e un puntatore fisico al blocco successivo. **Vantaggi:** Nessuna frammentazione esterna e allocazione dinamica immediata. **Svantaggi:** L'accesso diretto è inefficiente (bisogna scorrere la catena di blocchi); la perdita di un blocco compromette l'intero file.
 - *Allocazione a Indice:* Tutti i puntatori ai blocchi sparsi di un file vengono raccolti e centralizzati in un unico blocco speciale denominato **blocco indice**. **Vantaggi:** Supporta nativamente l'accesso diretto e azzerla la frammentazione esterna. **Svantaggi:** Comporta uno spreco di spazio sul disco per memorizzare i blocchi indice, soprattutto per i file molto piccoli.

15. Unix File System

Il File System di UNIX (UFS) organizza la memoria di massa suddividendo la partizione del disco in quattro regioni strutturali distinte e sequenziali:

1. **Boot Block:** Posizionato all'inizio del disco, ospita il codice eseguibile e le procedure necessarie all'inizializzazione del sistema durante la fase di bootstrap.
 2. **Super Block:** È il blocco di controllo della partizione; memorizza le dimensioni e i limiti geometrici delle altre regioni del disco, oltre a mantenere i puntatori alle liste dei blocchi dati liberi e degli i-node disponibili.
 3. **i-nodes (i-list):** Regione che ospita la tabella di tutti i descrittori dei file (gli i-node). Ad ogni singolo file corrisponde univocamente un solo i-node, identificato da un numero indice chiamato **i-number**.
 4. **Data Blocks:** È l'area più estesa del disco, contenente le effettive locazioni fisiche in cui vengono memorizzati i dati dei file e delle directory.
- **Struttura Interna dell'i-node:** L'i-node non contiene il nome del file, ma memorizza tutti i suoi metadati: tipo di file, UID del proprietario, GUID del gruppo, bit di permesso, timestamp temporali,

dimensione fisica e il **numero di link** (contatore di link fisici che puntano a quell'i-node; il file viene rimosso fisicamente dal disco solo quando questo contatore scende a zero).

- **Struttura dei Puntatori nell'i-node (Allocazione a indice UNIX):** Per mappare i blocchi dati sul disco, l'i-node implementa un sistema di puntamento ad albero per supportare file di ogni dimensione in modo efficiente:
 - *10 Puntatori Diretti:* Puntano direttamente ai primi 10 blocchi dati del file (garantiscono un accesso veloce in un singolo passo per file di piccole dimensioni).
 - *1 Puntatore Indiretto Singolo:* Punta a un blocco indice intermedio che contiene esclusivamente indirizzi di blocchi dati.
 - *1 Puntatore Indiretto Doppio:* Punta a un blocco indice di primo livello, il quale punta a blocchi indice di secondo livello, che a loro volta indirizzano i blocchi dati.
 - *1 Puntatore Indiretto Triplo:* Sviluppa la struttura su tre livelli gerarchici di blocchi indice, consentendo la gestione di file di enormi dimensioni.
- **Concetto di Directory in UNIX:** Una directory è un file speciale il cui contenuto all'interno dei Data Blocks è semplicemente una tabella composta da coppie: [i-number, file_name]. Fornisce la mappatura logica tra il nome visibile all'utente e il descrittore fisico reale del sistema.

16. Strutture dati del Kernel per accedere ai file

Quando un processo interagisce con i file, il Kernel UNIX coordina l'accesso attraverso tre strutture dati distinte e collegate a livelli gerarchici per ottimizzare le prestazioni e supportare la concorrenza:

1. **Tabella dei Descrittori di File (File Descriptor Table):** È una struttura dati privata, allocata internamente alla *User Structure* di ogni singolo processo. Contiene l'elenco di tutti i descrittori di file (fd) correntemente aperti dal processo. I primi tre indici di questa tabella sono riservati di default agli standard stream: 0 (Standard Input), 1 (Standard Output) e 2 (Standard Error). Ogni voce della tabella punta a una entry nella struttura successiva.
2. **Tabella dei File Aperti (Open File Table):** È una tabella globale, condivisa a livello di intero sistema. Ogni singola entry rappresenta una specifica operazione di apertura di un file (se uno stesso file viene aperto tre volte, anche dallo stesso processo, si generano tre entry distinte). Memorizza informazioni dinamiche legate alla sessione di utilizzo, come l'**I/O pointer** (l'offset che indica la posizione corrente di lettura/scrittura nel file) e la modalità di accesso selezionata (es. sola lettura, scrittura). Questa ridondanza permette a processi diversi di leggere o scrivere lo stesso file in punti indipendenti contemporaneamente. Ciascuna entry punta alla terza tabella.
3. **Tabella dei File Attivi (Active File Table o Inode Table):** È anch'essa una struttura dati globale del Kernel, che risiede stabilmente in RAM. Contiene una sola ed unica entry per ciascun file fisicamente aperto nel sistema, a prescindere da quanti processi lo stiano utilizzando. È di fatto una copia in memoria centrale dell'i-node del file memorizzato su disco. Conserva i metadati statici (permessi, dimensione) e i puntatori geometrici ai blocchi del disco, con lo scopo fondamentale di evitare continui ed onerosi accessi alla memoria secondaria per consultare gli attributi del file.

17. Ridirezione verso file (shell e system call), ridirezione verso pipe / piping (shell e system call)

- **Ridirezione verso File via Shell:** Si realizza utilizzando i metacaratteri > (scrittura distruttiva/sovrascrittura) o >> (scrittura in append) per l'output, e < per l'input. Es: `ls > file.txt`.
- **Ridirezione verso File via System Call:** All'interno del codice C, la ridirezione sfrutta il principio per cui il sistema assegna sempre il descrittore di file libero con l'indice più basso disponibile. Il procedimento standard prevede l'uso della chiamata `close()` combinata con `open()` o `dup2()`:

- Si invoca `close(1)` per chiudere intenzionalmente lo Standard Output del processo.
- Si esegue una system call `open("file.txt", ...)`. Essendo l'indice 1 appena liberato il più basso in tabella, il Kernel assegnerà automaticamente il descrittore 1 al nuovo file aperto.
- Qualsiasi successiva operazione di scrittura orientata all'output standard (es. `printf()`) scriverà direttamente nel file, poiché il flusso fa riferimento all'indice 1 della tabella dei descrittori.
- **Piping via Shell:** Si implementa interponendo il metacarattere barra verticale `|` tra due comandi autonomi (es. `ls | grep txt`). L'output del primo comando diventa l'input del secondo.
- **Piping via System Call:** Il meccanismo combina la creazione di un canale anonimo e la duplicazione dei descrittori mediante le chiamate `pipe()`, `fork()`, `close()` e `dup2()`:
 - Il processo padre invoca la system call `pipe(fd)`, ottenendo un array con due descrittori: `fd[0]` (estremità di lettura) e `fd[1]` (estremità di scrittura).
 - Viene eseguita una `fork()`; il processo figlio eredita fedelmente la stessa pipe e gli stessi descrittori aperti dal padre.
 - Per stabilire una comunicazione unidirezionale pulita, i due processi chiudono i canali non utilizzati:
 - Il processo scrittore (es. il Padre) esegue `close(fd[0])` (chiude la lettura), ridireziona il proprio Standard Output (1) sul canale di scrittura della pipe tramite `dup2(fd[1], 1)`, e scrive i dati.
 - Il processo lettore (es. il Figlio) esegue `close(fd[1])` (chiude la scrittura), ridireziona il proprio Standard Input (0) sul canale di lettura della pipe tramite `dup2(fd[0], 0)`, e si mette in ascolto tramite una chiamata di lettura.

18. Metodi di allocazione dell'immagine dei processi in memoria principale

Il sistema operativo deve decidere come mappare lo spazio di indirizzamento logico di un processo all'interno della memoria fisica (RAM). Gli appunti descrivono due approcci principali:

- **Allocazione Contigua (Modelli storici):** L'immagine del processo deve essere caricata in RAM in un blocco di locazioni fisicamente consecutive.
 - *Partizione Singola:* Utilizzata nei sistemi monoprogrammati; la memoria è rigidamente divisa in due sole aree fisiche: una riservata al codice del SO e l'altra interamente allocata al singolo processo utente attivo.
 - *Partizioni Multiple Fisse (MFT):* La RAM è permanentemente suddivisa in partizioni di dimensioni prestabilite in fase di boot. Ogni processo viene inserito in una partizione libera sufficientemente grande. **Limite:** Causa forte **frammentazione interna** (se un processo occupa meno spazio della dimensione della partizione, la memoria residua all'interno viene sprecata) e limita il grado di multiprogrammazione al numero fisso di partizioni.
 - *Partizioni Multiple Variabili (MVT):* Le partizioni vengono create dinamicamente dal SO ritagliando lo spazio esatto richiesto dal processo al momento del caricamento. Elimina la frammentazione interna, ma causa nel tempo **frammentazione esterna** (il continuo avvio e termine di processi frammenta la memoria libera in piccoli buchi non contigui isolati). Per risolvere il problema, il SO è costretto a eseguire periodicamente la **compattazione**, bloccando il sistema per spostare fisicamente i processi in RAM e riunire lo spazio vuoto.
- **Allocazione Non Contigua (Modelli Moderni):** L'immagine del processo può essere frammentata e distribuita in punti distanti della memoria fisica.

- **Paginazione:** Lo spazio di memoria fisica viene suddiviso in blocchi di dimensione fissa chiamati **frame**, mentre lo spazio logico del processo è segmentato in **pagine** della medesima dimensione. Le pagine di un processo possono essere allocate in frame non contigui ovunque vi sia spazio libero. Azzerata completamente la frammentazione esterna e riduce al minimo quella interna. La traduzione dinamica degli indirizzi logici in fisici è operata dal Kernel tramite la **tabella delle pagine**.
- **Segmentazione:** Lo spazio logico viene suddiviso in blocchi di lunghezza variabile detti **segmenti**, rispecchiando le unità semantiche del codice (es. segmento codice, segmento dati, segmento stack). Ogni segmento è allocato in RAM in modo contiguo, ma segmenti diversi dello stesso processo possono risiedere in zone separate. Soffre ancora di frammentazione esterna.
- **Segmentazione Paginata:** Tecnica ibrida moderna (adottata dalle architetture Intel e da Linux). Lo spazio logico del processo è organizzato in segmenti logici, i quali vengono a loro volta internamente suddivisi in pagine di dimensione fissa. Unisce i vantaggi di protezione della segmentazione all'efficienza di gestione della paginazione.

19. Memoria virtuale

- **Definizione:** È un'architettura di gestione della memoria supportata dai sistemi moderni che consente l'esecuzione di processi la cui immagine complessiva non si trova interamente caricata all'interno della memoria fisica centrale (RAM).
- **Principio di Funzionamento:** Permette al sistema operativo di aggirare il limite fisico della RAM disponibile, consentendo di lanciare programmi le cui dimensioni teoriche superano lo spazio hardware reale. Lo spazio di indirizzamento logico del processo è mappato sulla memoria secondaria (disco) e porzioni di esso vengono trasferite in RAM in modo selettivo solo ed esclusivamente quando l'architettura ne richiede l'esecuzione immediata.

20. Paginazione su richiesta

- **Definizione (Demand Paging):** È il meccanismo operativo principale su cui poggia l'intera architettura della memoria virtuale. Invece di caricare l'intero codice e i dati di un processo in RAM al momento dell'avvio, il sistema operativo carica inizialmente solo le pochissime pagine strettamente necessarie all'esecuzione delle prime istruzioni. Tutte le restanti pagine rimangono memorizzate all'interno del disco.
- **Il Meccanismo del Page Fault:** Quando il processo in esecuzione tenta di accedere a un indirizzo logico situato in una pagina che non è attualmente presente all'interno della RAM, l'hardware della MMU rileva l'assenza e genera un'interruzione hardware speciale denominata **Page Fault**.
- **Procedura di Gestione del Page Fault dal Kernel:**
 - Il controllo passa asincronamente al Kernel del SO.
 - Il Kernel verifica se la richiesta di accesso è lecita.
 - Se lecita, il SO individua la pagina richiesta sul disco.
 - Cerca un *frame* libero nella memoria fisica (RAM).
 - Se non ci sono frame fisici liberi, invoca gli algoritmi di scelta per identificare una pagina "vittima" da scaricare su disco.
 - Carica la pagina richiesta dal disco nel frame di RAM assegnato.
 - Aggiorna la tabella delle pagine del processo impostando il bit di validità.
 - Ripristina lo stato e istruisce la CPU di rieseguire l'istruzione esatta che aveva causato l'interruzione.

21. Sostituzione di pagine e identificazione di una vittima

- **Il Problema della Sovrallocazione:** Si verifica quando tutte le locazioni fisiche della RAM (i frame) sono interamente occupate da pagine di processi attivi e si manifesta un ulteriore Page Fault. Non essendoci spazio libero, il sistema operativo deve applicare una politica di sostituzione delle pagine.
- **Procedura:** Il Kernel seleziona una pagina attualmente residente in RAM, denominata **pagina vittima** ($SP_{\{vitt\}}$), esegue un'operazione di *swap-out* salvandola sul disco, dealloca il frame occupato e infine esegue lo *swap-in* caricando la nuova pagina richiesta ($SP_{\{new\}}$) aggiornando le tabelle di mappatura.
- **Algoritmi di Scelta della Vittima:**
 - **FIFO (First-In-First-Out):** Sostituisce la pagina che risiede in memoria fisica da più tempo in assoluto, ignorando completamente la frequenza con cui viene usata. È facile da implementare ma può penalizzare pagine pesantemente utilizzate dal sistema solo perché caricate all'inizio.
 - **LFU (Least Frequently Used):** Sceglie come vittima la pagina che è stata acceduta il minor numero di volte in un determinato intervallo temporale. Il tracciamento avviene incrementando un contatore di accessi hardware associato a ciascuna pagina.
 - **LRU (Least Recently Used):** Si basa sul principio di località temporale, assumendo che le pagine non utilizzate da molto tempo non verranno richieste nel breve futuro. Sostituisce la pagina che non viene acceduta da più tempo. È considerato un algoritmo eccellente, ma richiede un supporto hardware continuo e oneroso (registrazione di timestamp o strutture a stack ad ogni singolo accesso alla memoria), introducendo un overhead di sistema significativo.

22. Swapper vs pager

Entrambi sono componenti software del kernel deputati alla gestione della memoria e al trasferimento di dati tra la RAM e il disco, ma operano a livelli di granularità strutturale completamente diversi:

- **Swapper (Scheduler a Medio Termine):** Lavora a livello di **intero processo macroscopico**. Il suo compito principale è gestire la stabilità globale del sistema in caso di saturazione della memoria centrale, selezionando interi processi attivi (comprensivi di codice, dati, stack e PCB) per rimuoverli in blocco dalla RAM (*swap-out*) o reintrodurli (*swap-in*). Viene utilizzato per regolare il grado complessivo di multiprogrammazione.
- **Pager:** Rappresenta il motore interno della **memoria virtuale basata sulla paginazione su richiesta (demand paging)**. Opera a un livello di dettaglio estremamente più fine, gestendo esclusivamente il trasferimento di **singole pagine di memoria** alla volta. Viene spesso descritto formalmente come uno "swapper di pagine" poiché, anziché spostare l'intero blocco di un processo sul disco, monitora e sposta in modo chirurgico solo le specifiche porzioni di memoria (pagine) strettamente necessarie all'esecuzione istantanea.

23. Working set

- **Definizione:** È un modello teorico e operativo utilizzato dal sistema operativo per stimare e prevedere l'insieme preciso di pagine di cui un determinato processo ha assoluto bisogno in una specifica ed immediata finestra temporale.
- **Principio Logico:** Si basa sul principio di località di esecuzione dei programmi (un processo tende a concentrare i suoi accessi alla memoria su un ristretto sottoinsieme di codice e dati per un certo tempo).

- **Scopo:** Identificando il working set di un processo, il SO si impegna a caricare preventivamente e mantenere stabili in RAM tutte le pagine appartenenti a questo insieme. Questo pre-caricamento consente di ridurre in modo drastico i fenomeni di Page Fault a catena ed evita il collasso del sistema dovuto al *thrashing* (situazione in cui la CPU spreca tutto il tempo a scambiare pagine con il disco anziché far avanzare i processi).

24. Organizzazione logica e funzioni per la gestione dei dispositivi

- **Organizzazione Logica dell'I/O:** Ogni periferica o dispositivo hardware interfacciato al computer è dotato a livello hardware di due componenti chiave per dialogare con il sistema:
 - *Buffer di dispositivo:* Un'area di memoria locale integrata nel controller in cui vengono temporaneamente depositate le informazioni binarie lette o in procinto di essere scritte.
 - *Registri Speciali:* Registri di controllo e di stato in cui vengono salvate le specifiche e i parametri operativi delle funzioni da eseguire.
- **Procedura d'Interazione standard (Flusso delle Funzioni):** Quando un programma utente richiede un'operazione di lettura o scrittura su un dispositivo, l'interazione segue una sequenza guidata dal SO:
 - La CPU scrive nei registri speciali del controller del dispositivo coinvolto, impartendo il comando specifico.
 - Il controller hardware prende in carico l'operazione e trasferisce autonomamente i dati richiesti dal dispositivo fisico verso il proprio buffer locale.
 - A trasferimento completato, il controller invia un segnale di **interrupt asincrono alla CPU** per notificare la fine dell'operazione hardware.
 - I dati vengono infine trasferiti dal buffer del controller alla memoria centrale (RAM) del sistema per essere usati dal programma.
- **Tipologie di Gestione I/O:**
 - *I/O Sincrono:* Il processo che ha richiesto l'operazione viene rimosso dalla CPU e posto in stato di sospensione (*waiting/sleeping*) in attesa del completamento dell'operazione di I/O, bloccando la sua evoluzione finché i dati non sono pronti.
 - *I/O Asincrono:* Il sistema operativo avvia l'operazione ma restituisce immediatamente il controllo e la CPU al processo chiamante, consentendogli di proseguire l'esecuzione di altre istruzioni in parallelo e dando l'opportunità di lanciare più richieste di I/O contemporaneamente.

25. Politiche scheduling accesso Hard Disk

(Nota basata sulla logica degli algoritmi di scheduling forniti nei tuoi appunti applicati alla gestione delle code di richieste). Lo scheduling dell'Hard Disk ha l'obiettivo fondamentale di massimizzare il throughput complessivo delle operazioni di lettura/scrittura, minimizzando i tempi morti causati dagli spostamenti meccanici dei componenti (come la testina magnetica del disco) e scongiurando fenomeni di Starvation.

- Le politiche ordinano la coda delle richieste pendenti secondo parametri precisi:
 - **FIFO (First-In-First-Out) / FCFS:** Le richieste di accesso ai blocchi del disco vengono servite in base al puro ordine cronologico di sottomissione. È equa e semplice da implementare, ma altamente inefficiente dal punto di vista meccanico poiché può costringere la testina a continui spostamenti da un'estremità all'altra del disco.
 - **SSTF (Shortest Seek Time First - assimilabile al concetto di prioritizzazione del più corto / SNPF degli appunti):** Il sistema seleziona tra le richieste in coda quella posizionata sul blocco fisicamente più vicino alla posizione corrente della testina del disco. Questo approccio

minimizza i tempi di setup e di posizionamento meccanico, incrementando la velocità di I/O, ma espone il sistema al rischio di Starvation per le richieste posizionate su tracce lontane se continuano ad arrivare richieste vicine.

26. RAID: Redundant Array of Independent Disks

Il RAID è una tecnologia architetturale che consente di connettere ed utilizzare in parallelo più dischi fissi indipendenti, coordinati dal sistema come se fossero un'unica unità logica. È progettata per perseguire tre obiettivi chiave: incrementare le prestazioni in lettura/scrittura (tramite accessi I/O paralleli sui canali), aumentare l'affidabilità generale e garantire la tolleranza ai guasti dell'hardware mediante la ridondanza strutturale dei dati.

Gli appunti descrivono quattro livelli principali di configurazione:

- **RAID Livello 0 (Striping):** I dati vengono suddivisi e distribuiti in parallelo su tutti i dischi dell'array. **Vantaggi:** Aumenta le prestazioni di I/O, poiché i segmenti di un file possono essere letti o scritti contemporaneamente sfruttando più canali in parallelo. **Svantaggi:** Non offre alcuna ridondanza né protezione contro i guasti; il danneggiamento di un singolo disco distrugge l'intero archivio dati.
- **RAID Livello 1 (Mirroring):** Ogni singolo blocco di dati scritto nel sistema viene duplicato e replicato fedelmente su due dischi distinti speculari. **Vantaggi:** Garantisce un'elevatissima tolleranza ai guasti (se un disco si rompe, l'altro contiene la copia esatta pronta all'uso). **Svantaggi:** Comporta un costo di memorizzazione elevato, rendendo disponibile all'utente finale soltanto il 50% dello spazio totale dei dischi acquistati.
- **RAID Livello 5 (Parità Distribuita):** I dati vengono distribuiti sui dischi alternati a sezioni speciali di **parità**. Il blocco di parità viene calcolato applicando l'operazione logica bit a bit **XOR** (OR esclusivo) sui corrispondenti blocchi di dati. La parità non è isolata su un unico disco ma è distribuita ciclicamente su tutti i dischi dell'array. In caso di guasto hardware di un singolo disco, il sistema è in grado di ricostruire matematicamente le informazioni andate perse in tempo reale combinando i dati rimasti con la sezione di parità. Offre un ottimo compromesso tra sicurezza e spazio disponibile.
- **RAID Livello 6 (Doppia Parità Distribuita):** Evoluzione del RAID 5, implementa due blocchi di parità distinti e indipendenti distribuiti su tutti i dischi del sistema. **Vantaggi:** Incrementa la resilienza del sistema in modo estremo, consentendo all'array di sopravvivere e continuare a funzionare anche in caso di guasto hardware contemporaneo di due dischi. **Svantaggi:** Le operazioni di scrittura risultano computazionalmente più onerose e lente rispetto al RAID 5, a causa della necessità di calcolare ed aggiornare due volte le formule matematiche della parità ad ogni singola modifica dei dati.

27. Protezione

- **Isolamento della Memoria:** All'interno dei sistemi multiprogrammati o time-sharing, ogni processo possiede uno spazio di indirizzamento logico privato. È compito tassativo del sistema operativo impedire in modo assoluto che un programma in esecuzione possa accedere, intenzionalmente o per errore, ad aree di memoria esterne al proprio spazio o appartenenti ad altri processi o al Kernel stesso.
- **Architettura Dual-Mode (Duplice modo di funzionamento):** Per garantire la sicurezza ed evitare che programmi utente eseguano istruzioni dannose per l'hardware, le CPU moderne implementano una protezione a livello hardware basata su due modalità operative, distinte da un bit di controllo:

- **User Mode (Modalità Utente - Bit impostato a 1):** È la modalità limitata in cui vengono eseguiti i normali programmi applicativi dell'utente. In questa modalità non è consentito l'accesso diretto all'hardware o l'esecuzione di istruzioni privilegiate.
- **Kernel Mode (Modalità Supervisore/Privilegiata - Bit impostato a 0):** È la modalità protetta in cui ha facoltà di operare esclusivamente il nucleo del Sistema Operativo. Consente l'accesso illimitato a tutte le risorse fisiche della macchina e l'esecuzione di qualsiasi istruzione hardware.
- **Il Meccanismo di Passaggio (System Call):** Se un programma utente in esecuzione in *User Mode* necessita di compiere un'operazione privilegiata o potenzialmente pericolosa (es. leggere un file dal disco o allocare memoria), non può farlo direttamente. Deve generare un'interruzione software invocando una **System Call**. Questo commuta istantaneamente il bit di controllo della CPU in *Kernel Mode*, trasferendo lo stato e il comando al SO che prenderà in carico l'operazione in sicurezza, per poi reimpostare il sistema in *User Mode* al termine restituendo il controllo al programma.